



ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL

Práctica grupal: Alineación de cadenas

Programación dinámica

Integrantes

Sebastián Escobar

Luis Mendoza

Guillermo Terán

Paralelo

1-101

Profesor/a

Tapia Rosero Ana Teresa

Fecha

lunes 17 de agosto del 2026

Guayaquil - Ecuador

1. Objetivo de la práctica

Implementar en Python un algoritmo de alineación de cadenas usando programación dinámica. El programa debe recibir dos cadenas, construir la matriz de costos, mostrarla en pantalla y obtener una alineación con el costo mínimo.

2. Descripción de la solución

El programa trabaja con dos cadenas ingresadas por teclado. Para cada posición se calcula el menor costo posible considerando tres alternativas: comparar o sustituir caracteres, dejar un espacio en la segunda cadena o dejar un espacio en la primera cadena.

La primera fila y la primera columna de la matriz se inicializan con el costo acumulado de los espacios. Luego se completa el resto de la matriz desde la esquina superior izquierda hasta llegar a la última celda.

Finalmente se recorre la matriz desde el final hacia el inicio para reconstruir una alineación posible y mostrar las operaciones encontradas.

3. Partes principales del Código

- **crear_matriz ()** crea e inicializa la matriz y calcula el costo mínimo para cada subproblema.
- **mostrar_matriz ()** imprime la matriz con las letras de las dos cadenas como encabezados.
- **reconstruir_alineacion ()** recorre la matriz en sentido contrario para formar las cadenas alineadas y registrar las operaciones.
- **resolver_alineacion ()** une los pasos anteriores y presenta el resultado final al usuario.

4. Cambios realizados al código inicial

- Se permite ingresar las dos cadenas desde la consola en lugar de trabajar solamente con valores fijos.
- Se agregó la impresión de la matriz de programación dinámica para poder revisar cómo se obtiene el resultado.
- Se reconstruye una alineación y se muestran las operaciones de sustitución, inserción o borrado.
- Se utilizaron listas normales de Python, por lo que el programa no necesita instalar NumPy para ejecutarse.

5. Análisis del algoritmo

Si la primera cadena tiene longitud m y la segunda longitud n , la matriz contiene $(m + 1)(n + 1)$ posiciones. Cada posición se calcula una sola vez y se realizan comparaciones de costo constante.

Tiempo de ejecución: $O(m \cdot n)$

Memoria utilizada: $O(m \cdot n)$, porque se guarda la matriz completa para poder mostrarla y reconstruir la alineación.

6.1 Prueba 1

Cadena 1: casa

Cadena 2: casa

Costo mínimo obtenido: 0

Operaciones observadas: ninguno

Captura de pantalla:

```
ALINEACION DE CADENAS - PROGRAMACION DINAMICA
Las penalidades se dejan en 1 para que la prueba sea facil de revisar.

Ingrese la primera cadena: casa
Ingrese la segunda cadena: casa

Matriz resultante:

  -   C   A   S   A
-   0   1   2   3   4
C   1   0   1   2   3
A   2   1   0   1   2
S   3   2   1   0   1
A   4   3   2   1   0

Resultado:
Cadena 1: CASA
Cadena 2: CASA
Costo minimo: 0

Operaciones encontradas:
No se necesitan cambios.
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code> |
```

Comentario breve de la prueba:

Esta prueba confirma que el algoritmo funciona correctamente cuando las cadenas son idénticas. El costo mínimo es 0, lo cual es correcto porque no se requiere ninguna operación de edición. La matriz muestra una diagonal de ceros, evidenciando que cada carácter coincide perfectamente. El programa identifica correctamente que no se necesitan cambios y no muestra operaciones. Esta prueba sirve como caso base para validar que el algoritmo maneja adecuadamente el escenario más simple.

6.2 Prueba 2

Cadena 1: gato

Cadena 2: perro

Costo mínimo obtenido: 4

Operaciones observadas:

1. Insertar P
2. Sustituir G por E
3. Sustituir A por R
4. Sustituir T por R

Captura de pantalla:

```
ALINEACION DE CADENAS - PROGRAMACION DINAMICA
Las penalidades se dejan en 1 para que la prueba sea facil de revisar.

Ingrese la primera cadena: gato
Ingrese la segunda cadena: perro

Matriz resultante:

  -   P   E   R   R   O
-  0   1   2   3   4   5
G  1   1   2   3   4   5
A  2   2   2   3   4   5
T  3   3   3   3   4   5
O  4   4   4   4   4   4

Resultado:
Cadena 1: -GATO
Cadena 2: PERRO
Costo mínimo: 4

Operaciones encontradas:
1. Insertar P
2. Sustituir G por E
3. Sustituir A por R
4. Sustituir T por R
```

Comentario breve de la prueba:

Esta prueba evalúa el algoritmo con cadenas completamente diferentes: "GATO" y "PERRO". El costo mínimo obtenido es 4, lo cual es razonable considerando que la palabra "GATO" tiene 4 letras y "PERRO" tiene 5. El programa identifica correctamente la necesidad de insertar la letra "P" al inicio y realizar tres sustituciones para transformar "GATO" en "PERRO". La matriz muestra el costo acumulado de manera ordenada y la reconstrucción de la alineación es consistente con las operaciones mostradas. Se comprueba que el algoritmo maneja adecuadamente casos con sustituciones y gaps.

6.3 Prueba 3

Cadena 1: algoritmo

Cadena 2: logaritmo

Costo mínimo obtenido: 3

Operaciones observadas: 1. Sustituir A por L
2. Sustituir L por O
3. Sustituir O por A

Captura de pantalla:

```
ALINEACION DE CADENAS - PROGRAMACION DINAMICA
Las penalidades se dejan en 1 para que la prueba sea facil de revisar.

Ingrese la primera cadena: algoritmo
Ingrese la segunda cadena: logaritmo

Matriz resultante:

  -   L   O   G   A   R   I   T   M   O
- 0  1  2  3  4  5  6  7  8  9
A 1  1  2  3  3  4  5  6  7  8
L 2  1  2  3  4  4  5  6  7  8
G 3  2  2  2  3  4  5  6  7  8
O 4  3  2  3  3  4  5  6  7  7
R 5  4  3  3  4  3  4  5  6  7
I 6  5  4  4  4  4  3  4  5  6
T 7  6  5  5  5  5  4  3  4  5
M 8  7  6  6  6  6  5  4  3  4
O 9  8  7  7  7  7  6  5  4  3

Resultado:
Cadena 1: ALGORITMO
Cadena 2: LOGARITMO
Costo mínimo: 3

Operaciones encontradas:
1. Sustituir A por L
2. Sustituir L por O
3. Sustituir O por A
```

Comentario breve de la prueba:

Esta prueba representa un caso más complejo donde las cadenas "ALGORITMO" y "LOGARITMO" comparten muchas letras pero están desalineadas. El costo mínimo obtenido es 3, lo cual es correcto ya que se necesitan tres sustituciones ($A \rightarrow L$, $L \rightarrow O$, $O \rightarrow A$) para transformar una palabra en otra. El algoritmo logra encontrar la mejor alineación posible, demostrando su capacidad para manejar casos donde es necesario evaluar múltiples combinaciones de operaciones. La matriz de costos refleja adecuadamente todas las posibilidades y el programa muestra correctamente las operaciones necesarias.

6.4 Prueba 4

Cadena 1: ATGCTAGCTAGCTACGATCGATCGTAGCTAGCTAGCTAG

Cadena 2: ATGCTAGCTAGATACGATCGTTTCGTAGCTAGCTAGCTAG

Costo mínimo obtenido: 6

Operaciones observadas: 1. Sustituir C por A

2. Sustituir A por T

Captura de pantalla:

```
C:\Users\guill\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\guill\PycharmProjects\WelcomeScreen\alineaacion_matriz_inicial.py
Matriz de programacion dinamica para x='ATGCTAGCTAGCTACGATCGATCGTAGCTAGCTAGCTAG', y='ATGCTAGCTAGATACGATCGTTTCGTAGCTAGCTAGCTAG':
- A T G C T A G C T A G A T A C G A T C G T T C G T A G C T A G C T A G
- 0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66 68 70 72 74 76 78
A 2 0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66 68 70 72 74 76
T 4 2 0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66 68 70 72 74
G 6 4 2 0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66 68 70 72
C 8 6 4 2 0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66 68 70
T 10 8 6 4 2 0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66 68
A 12 10 8 6 4 2 0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66
G 14 12 10 8 6 4 2 0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64
C 16 14 12 10 8 6 4 2 0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62
T 18 16 14 12 10 8 6 4 2 0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60
A 20 18 16 14 12 10 8 6 4 2 0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58
G 22 20 18 16 14 12 10 8 6 4 2 0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56
C 24 22 20 18 16 14 12 10 8 6 4 2 3 5 7 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54
T 26 24 22 20 18 16 14 12 10 8 6 4 5 3 5 7 9 11 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52
A 28 26 24 22 20 18 16 14 12 10 8 6 4 5 3 5 7 9 11 13 15 17 19 21 23 25 24 26 28 30 32 34 36 38 40 42 44 46 48 50
C 30 28 26 24 22 20 18 16 14 12 10 8 6 7 5 3 5 7 9 11 13 15 17 19 21 23 25 27 26 28 30 32 34 36 38 40 42 44 46 48
G 32 30 28 26 24 22 20 18 16 14 12 10 8 9 7 5 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 30 32 34 36 38 40 42 44 46
A 34 32 30 28 26 24 22 20 18 16 14 12 10 11 9 7 5 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 34 36 38 40 42 44
T 36 34 32 30 28 26 24 22 20 18 16 14 12 10 11 9 7 5 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 38 40 42
C 38 36 34 32 30 28 26 24 22 20 18 16 14 12 13 11 9 7 5 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43
G 40 38 36 34 32 30 28 26 24 22 20 18 16 14 15 13 11 9 7 5 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41
A 42 40 38 36 34 32 30 28 26 24 22 20 18 16 14 15 13 11 9 7 5 6 8 10 12 14 13 15 17 19 21 23 25 27 29 31 33 35 37 39
T 44 42 40 38 36 34 32 30 28 26 24 22 20 18 16 17 15 13 11 9 7 5 6 8 10 12 14 16 18 17 19 21 23 25 27 29 31 33 35 37
C 46 44 42 40 38 36 34 32 30 28 26 24 22 20 18 16 17 15 13 11 9 7 8 6 8 10 12 14 16 18 20 22 21 23 25 27 29 31 33 35
G 48 46 44 42 40 38 36 34 32 30 28 26 24 22 20 18 16 17 15 13 11 9 10 8 6 8 10 12 14 16 18 20 22 24 26 25 27 29 31 33
T 50 48 46 44 42 40 38 36 34 32 30 28 26 24 22 20 18 19 17 15 13 11 9 10 8 6 8 10 12 14 16 18 20 22 24 26 28 27 29 31
A 52 50 48 46 44 42 40 38 36 34 32 30 28 26 24 22 20 18 19 17 15 13 11 12 10 8 6 8 10 12 14 16 18 20 22 24 26 28 27 29
G 54 52 50 48 46 44 42 40 38 36 34 32 30 28 26 24 22 20 21 19 17 15 13 14 12 10 8 6 8 10 12 14 16 18 20 22 24 26 28 27
C 56 54 52 50 48 46 44 42 40 38 36 34 32 30 28 26 24 22 23 21 19 17 15 13 14 12 10 8 6 8 10 12 14 16 18 20 22 24 26 28
T 58 56 54 52 50 48 46 44 42 40 38 36 34 32 30 28 26 24 22 23 21 19 17 15 16 14 12 10 8 6 8 10 12 14 16 18 20 22 24 26
A 60 58 56 54 52 50 48 46 44 42 40 38 36 34 32 30 28 26 24 25 23 21 19 17 18 16 14 12 10 8 6 8 10 12 14 16 18 20 22 24
G 62 60 58 56 54 52 50 48 46 44 42 40 38 36 34 32 30 28 26 27 25 23 21 19 17 18 16 14 12 10 8 6 8 10 12 14 16 18 20 22
C 64 62 60 58 56 54 52 50 48 46 44 42 40 38 36 34 32 30 28 26 27 25 23 21 19 20 18 16 14 12 10 8 6 8 10 12 14 16 18 20
T 66 64 62 60 58 56 54 52 50 48 46 44 42 40 38 36 34 32 30 28 29 27 25 23 21 19 20 18 16 14 12 10 8 6 8 10 12 14 16 18
A 68 66 64 62 60 58 56 54 52 50 48 46 44 42 40 38 36 34 32 30 31 29 27 25 23 21 19 20 18 16 14 12 10 8 6 8 10 12 14 16
G 70 68 66 64 62 60 58 56 54 52 50 48 46 44 42 40 38 36 34 32 30 31 29 27 25 23 21 19 20 18 16 14 12 10 8 6 8 10 12 14
C 72 70 68 66 64 62 60 58 56 54 52 50 48 46 44 42 40 38 36 34 32 33 31 29 27 25 23 21 19 20 18 16 14 12 10 8 6 8 10 12
T 74 72 70 68 66 64 62 60 58 56 54 52 50 48 46 44 42 40 38 36 34 32 33 31 29 27 25 23 21 19 20 18 16 14 12 10 8 6 8 10
A 76 74 72 70 68 66 64 62 60 58 56 54 52 50 48 46 44 42 40 38 36 34 35 33 31 29 27 25 23 21 19 20 18 16 14 12 10 8 6 8
G 78 76 74 72 70 68 66 64 62 60 58 56 54 52 50 48 46 44 42 40 38 36 37 35 33 31 29 27 25 23 21 19 20 18 16 14 12 10 8 6

Minimum Penalty in aligning the genes = 6
The aligned genes are:
X seq: ATGCTAGCTAGCTACGATCGATCGTAGCTAGCTAGCTAG
Y seq: ATGCTAGCTAGATACGATCGTTTCGTAGCTAGCTAGCTAG

Process finished with exit code 0
```

Comentario breve de la prueba:

Esta prueba utiliza dos secuencias mucho más largas que las anteriores (39 caracteres cada una), similares a las que se manejan en bioinformática real. El costo mínimo obtenido es 6, correspondiente a dos sustituciones puntuales (C por A en la posición 12 y A por T en la posición 21); en las 37 posiciones restantes las cadenas coinciden exactamente, por lo que no fue necesario insertar ningún espacio. Esto es coherente porque, al ser las secuencias tan parecidas entre sí, alinear carácter por carácter resulta más económico que desplazar alguna de las cadenas con gaps. Esta prueba permite comprobar que el algoritmo también funciona correctamente y de forma eficiente con entradas de mayor tamaño, generando sin problemas una matriz de 40×40 posiciones, y que el costo mínimo mostrado en pantalla coincide exactamente con el valor de la última celda de la matriz.

6.5 Prueba 5

Cadena 1: PROGRAMACION

Cadena 2: PROGAMACION

```

C:\Users\sebas\AppData\Local\Programs\Python\Python312\python.exe C:\Users\sebas\OneDrive\Escritorio\alineacion_cadenas.py
ALINEACION DE CADENAS - PROGRAMACION DINAMICA
Las penalidades se dejan en 1 para que la prueba sea facil de revisar.

Ingrese la primera cadena: PROGRAMACION
Ingrese la segunda cadena: PROGAMACION

Matriz resultante:

  - P R O G A M A C I O N
- 0 1 2 3 4 5 6 7 8 9 10 11
P 1 0 1 2 3 4 5 6 7 8 9 10
R 2 1 0 1 2 3 4 5 6 7 8 9
O 3 2 1 0 1 2 3 4 5 6 7 8
G 4 3 2 1 0 1 2 3 4 5 6 7
R 5 4 3 2 1 1 2 3 4 5 6 7
A 6 5 4 3 2 1 2 2 3 4 5 6
M 7 6 5 4 3 2 1 2 3 4 5 6
A 8 7 6 5 4 3 2 1 2 3 4 5
C 9 8 7 6 5 4 3 2 1 2 3 4
I 10 9 8 7 6 5 4 3 2 1 2 3
O 11 10 9 8 7 6 5 4 3 2 1 2
N 12 11 10 9 8 7 6 5 4 3 2 1

Resultado:
Cadena 1: PROGRAMACION
Cadena 2: PROG-AMACION
Costo minimo: 1

Operaciones encontradas:
1. Borrar R

Process finished with exit code 0

```

Resultado: Cadena 1: PROGRAMACION Cadena 2: PROG-AMACION

Costo mínimo: 1

Operaciones encontradas: 1. Borrar R

En esta prueba se ingresaron dos cadenas muy parecidas, pero la segunda tiene un carácter menos. Se utilizó este caso para comprobar que el algoritmo pueda reconocer la necesidad de insertar un espacio en la alineación y no solamente realizar sustituciones.

7. Conclusiones

Luego de realizar las pruebas se pudo comprobar que el algoritmo construye la matriz de costos y obtiene una alineación de las cadenas ingresadas. El uso de programación dinámica evita volver a calcular los mismos subproblemas y permite obtener el costo mínimo de forma ordenada.

Las pruebas también permiten verificar visualmente que la última posición de la matriz coincide con el costo mostrado por el programa y que las operaciones obtenidas corresponden con la alineación presentada.

Observación del grupo

Durante el desarrollo de la práctica, notamos que la elección de la función de costo (`penalidad_diferencia=1`, `penalidad_gap=1`) es crucial para obtener resultados coherentes. Si se modifican estos valores, el algoritmo podría favorecer más sustituciones o más gaps, lo que afectaría la alineación final. Una posible mejora para el programa sería permitir al usuario personalizar estas penalidades, ya que en aplicaciones reales los costos de sustitución y gap suelen ser diferentes. Además, se podría optimizar el uso de memoria implementando una versión que solo almacene dos filas de la matriz, reduciendo la complejidad espacial a $O(\min(m,n))$ sin afectar el resultado final.